

EXP10: A* Algorithm

Name: Y. Roshitha (192424400)

Aim:

To develop a Python program that implements the A* Algorithm to find the shortest path from a start node to a goal node using both path cost and heuristic estimates.

Algorithm:

Step 1: Represent the graph using an adjacency list along with edge costs, and define a heuristic value for every node.

Step 2: Initialize the open list with the start node, having $f(n) = g(n) + h(n)$, where $g(n) = 0$ initially.

Step 3: If the open list is empty, report that no path exists.

Step 4: Otherwise, select the node with the lowest $f(n)$ value from the open list.

Step 5: If the selected node is the goal node, return the path and total cost.

Step 6: Otherwise, expand its neighbours, compute their g , h and f values, and add them to the open list.

Step 7: Mark the current node as visited and repeat Steps 3 to 6.

Step 8: Display the final path and total cost once the goal is reached.

Source Code:

```
# Program to implement A* Algorithm
```

```
import heapq
```

```
def a_star(graph, heuristics, start, goal):
```

```
    open_list = [(heuristics[start], 0, start, [start])]
```

```
    visited = set()
```

```
    while open_list:
```

```
        f, g, node, path = heapq.heappop(open_list)
```

```
        if node == goal:
```

```
            return path, g
```

```
        if node in visited:
```

```
            continue
```

```
        visited.add(node)
```

```
        for neighbour, cost in graph[node]:
```

```
            if neighbour not in visited:
```

```
                new_g = g + cost
```

```
                new_f = new_g + heuristics[neighbour]
```

```
                heapq.heappush(open_list, (new_f, new_g, neighbour, path + [neighbour]))
```

```
return None, float('inf')
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('A', 1), ('D', 2), ('E', 5)],  
    'C': [('A', 4), ('F', 3)],  
    'D': [('B', 2), ('G', 6)],  
    'E': [('B', 5), ('G', 2)],  
    'F': [('C', 3), ('G', 1)],  
    'G': []  
}
```

```
heuristics = {'A': 6, 'B': 4, 'C': 4, 'D': 2, 'E': 2, 'F': 1, 'G': 0}
```

```
start_node = 'A'
```

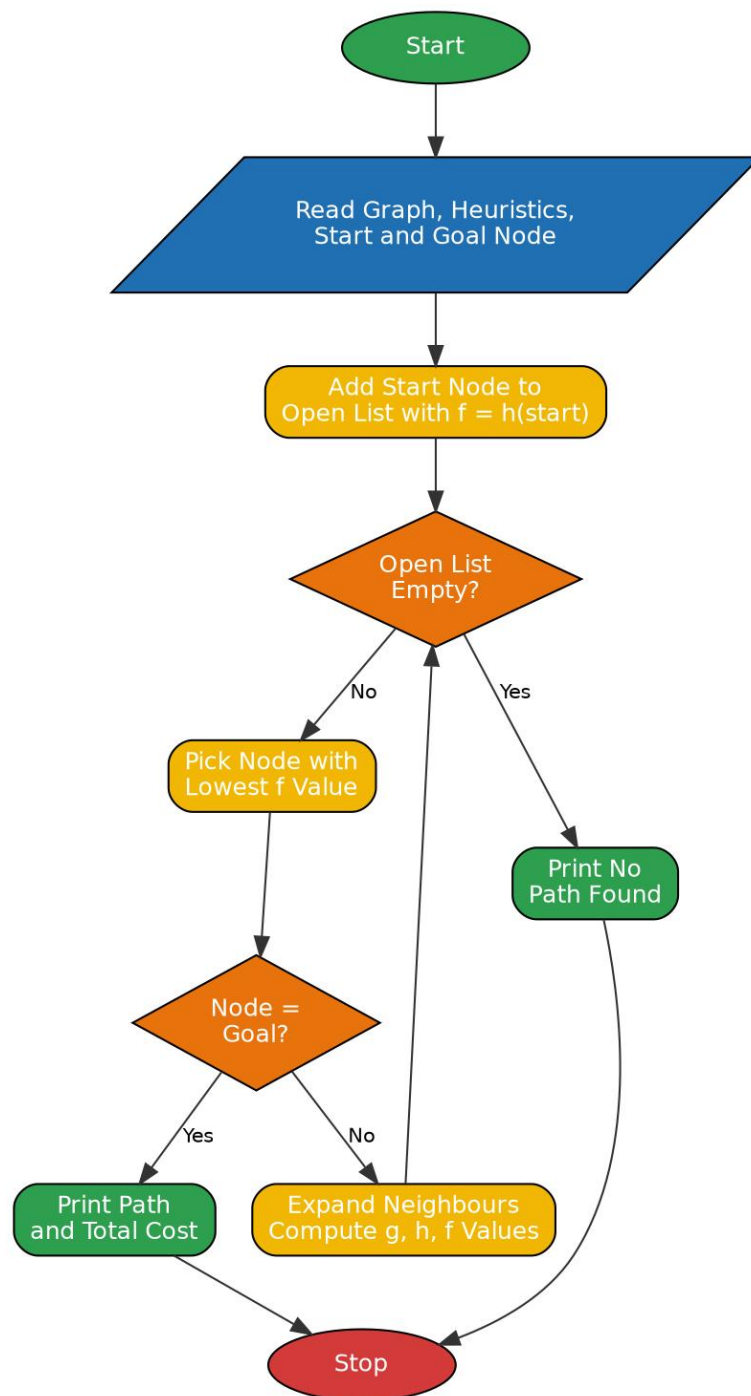
```
goal_node = 'G'
```

```
path, cost = a_star(graph, heuristics, start_node, goal_node)
```

```
print("Path found:", " -> ".join(path))
```

```
print("Total cost:", cost)
```

Flowchart:



Output:

```
>>> ===== RESTART: C:/Users/yanar/Documents/CSA1717/EXP10 =====>>>
Path found: A -> B -> E -> G
Total cost: 8
>>>
```

Result:

Thus the program to implement the A* Algorithm was written, executed successfully and the output was verified.